C# and SQL

Using C# to manage SQL databases.

Última atualização a *11 de maio de 2020*

Conexão

Abrir conexão

> **SqlConnection** (System.Data.SqlClient) +
>
> ***Construtores***
>
> SqlConnection(String)
>
> SqlConnection(String, Sql Credential)
>
> ***Propriedades***
>
> State — Estado da ligação do tipo ConnectionState (System.Data)
>
> *Mais comuns são Open, Closed, Broken*
>
> ***Métodos***
>
> Open() — Abre a conexão
>
> Close() — Fecha a conexão

```
using System.Data.SqlClient;
using System.Data;

SqlConnectioc cn = new SqlConnection("data source=DESKTOP-1BLL8VH;integrated
security=true;initial catalog=Northwind");
cn.Open();
if(cn.State==ConnectionState.Open)
    MessageBox.Show("Sucessfully connected to DB!");
cn.Close();
```

Comandos

Para realizar comando sobre a BD utiliza-se a classe **SqlCommand**

> **SqlCommand** (System.Data.SqlClient) +

Constructores

SqlCommand(String, Sql Connection) The String is the command to execute

Propriedades

CommandText Texto do comando a executar

Parameters Devolve elemento do tipo SqlParameterCollection

Connection Conexão

Métodos

ExecuteReader() Executa o comando e devolve um SqlDataReader

ExecuteNonQuery() Executa comando e devolve o número de linhas afetadas

Operações de leitura

SqlDataReader (System.Data.SqlClient) +

Propriedades

FieldCount Número de colunas da linha atual

HasRows Devolve valor de verdade

[] Devolve valor da coluna (nome) dada na linha atual

[] Devolve valor da coluna (índice) dada na linha atual

Métodos

Read() Avança para a linha seguinte

Close()

```
using System.Data.SqlClient;

SqlCommand cmd = new SqlCommand("SELECT * FROM TABLE", cn);
SqlDataReader reader = cmd.ExecuteReader();
while(reader.Read())
{
    String name = reader["name"].ToString();
    // Supposing that there is an attribute called 'name'
}
```

Operações de escrita

SqlParameterCollection (System.Data.SqlClient) +

Métodos

Clear() Remove todos os objetos SqlParameter da coleção

AddWithValue(String, Object) Adiciona valor ao fim da coleção

```
using System.Data.SqlClient;

// Create command
SqlCommand cmd = new SqlCommand();
cmd.CommandText = "INSERT Table(atr1, atr2) VALUES (@val1, @val2)";
cmd.Parameters.Clear();
cmd.Parameters.AddWithValue("@val1", "<val>");
cmd.Parameters.AddWithValue("@val2", "<val>");
cmd.Connection = cn;
// Execute command
try {
    cmd.ExecuteNonQuery();
}
catch (Exception ex)
{
    throw new Exception("Failed to update contact in database. \n ERROR MESSAGE:
\n" + ex.Message);
}
```

> Outras operações de escrita
>
> ```
> UPDATE <table> SET <attr>=<value>, ... WHERE <condition>;
> DELETE FROM <table> WHERE <condition>;
> ```

© Gonçalo Matos, 92972, LEI (Universidade de Aveiro), 2020